

Employee Attrition Prediction

CodTech Data Science Internship — Task 2

A comprehensive analysis of IBM's HR Analytics dataset to identify the key factors driving employee attrition and build a machine learning model to predict which employees are at risk of leaving.

Notebook Structure:

1. Import Libraries
2. Load Dataset
3. Data Overview
4. Data Cleaning & Preprocessing
5. Exploratory Data Analysis (EDA)
6. Feature Engineering
7. Model Preparation
8. Gradient Boosting Classifier
9. Model Evaluation
10. Conclusions

```
In [1]: # — 1. Import Libraries —————
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.ticker as mtick
import seaborn as sns

from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (accuracy_score, classification_report,
                             confusion_matrix, roc_curve, auc)
from sklearn.preprocessing import LabelEncoder

import warnings
warnings.filterwarnings('ignore')

sns.set_theme(style='darkgrid')
plt.rcParams.update({
    'figure.figsize': (10, 6),
    'axes.titlesize': 14,
    'axes.titleweight': 'bold'
})

print("All libraries imported successfully.")
```

All libraries imported successfully.

2. Load Dataset

The IBM HR Analytics Employee Attrition dataset is loaded directly from a public GitHub repository — no manual download required.

Dataset: 1,470 employee records | 35 features

Target: Attrition — whether an employee left the company (Yes/No)

```
In [2]: # Load dataset
import pandas as pd

df = pd.read_csv('IBM_HR_Employee_Attrition.csv')

print("Dataset shape:", df.shape)
print("\nAttrition distribution:")
print(df['Attrition'].value_counts())
print("\nFirst few rows:")
df.head(10)
```

Dataset shape: (1470, 35)

Attrition distribution:

Attrition

No 1233

Yes 237

Name: count, dtype: int64

First few rows:

```
Out[2]:
```

	Age	Attrition	BusinessTravel	DailyRate	Department	DistanceFromHome	...
0	41	Yes	Travel_Rarely	1102	Sales	1	
1	49	No	Travel_Frequently	279	Research & Development	8	
2	37	Yes	Travel_Rarely	1373	Research & Development	2	
3	33	No	Travel_Frequently	1392	Research & Development	3	
4	27	No	Travel_Rarely	591	Research & Development	2	
5	32	No	Travel_Frequently	1005	Research & Development	2	
6	59	No	Travel_Rarely	1324	Research & Development	3	
7	30	No	Travel_Rarely	1358	Research & Development	24	
8	38	No	Travel_Frequently	216	Research & Development	23	
9	36	No	Travel_Rarely	1299	Research & Development	27	

10 rows × 35 columns

3. Data Overview

Examining data types, statistics, and checking data quality.

```
In [3]: # — 3a. Dataset Info —————
print("=== Dataset Info ===")
df.info()

=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                                Non-Null Count  Dtype
---  -
 0   Age                                  1470 non-null   int64
 1   Attrition                           1470 non-null   object
 2   BusinessTravel                       1470 non-null   object
 3   DailyRate                           1470 non-null   int64
 4   Department                           1470 non-null   object
 5   DistanceFromHome                    1470 non-null   int64
 6   Education                           1470 non-null   int64
 7   EducationField                       1470 non-null   object
 8   EmployeeCount                       1470 non-null   int64
 9   EmployeeNumber                      1470 non-null   int64
10   EnvironmentSatisfaction              1470 non-null   int64
11   Gender                               1470 non-null   object
12   HourlyRate                           1470 non-null   int64
13   JobInvolvement                       1470 non-null   int64
14   JobLevel                             1470 non-null   int64
15   JobRole                              1470 non-null   object
16   JobSatisfaction                      1470 non-null   int64
17   MaritalStatus                       1470 non-null   object
18   MonthlyIncome                       1470 non-null   int64
19   MonthlyRate                         1470 non-null   int64
20   NumCompaniesWorked                  1470 non-null   int64
21   Over18                              1470 non-null   object
22   OverTime                            1470 non-null   object
23   PercentSalaryHike                   1470 non-null   int64
24   PerformanceRating                   1470 non-null   int64
25   RelationshipSatisfaction              1470 non-null   int64
26   StandardHours                       1470 non-null   int64
27   StockOptionLevel                    1470 non-null   int64
28   TotalWorkingYears                   1470 non-null   int64
29   TrainingTimesLastYear               1470 non-null   int64
30   WorkLifeBalance                     1470 non-null   int64
31   YearsAtCompany                      1470 non-null   int64
32   YearsInCurrentRole                  1470 non-null   int64
33   YearsSinceLastPromotion              1470 non-null   int64
34   YearsWithCurrManager                 1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

```
In [4]: # — 3b. Statistical Summary (Numeric Features) —————
df.describe()
```

Out[4]:

	Age	DailyRate	DistanceFromHome	Education	EmployeeCount
count	1470.000000	1470.000000	1470.000000	1470.000000	1470.0
mean	36.923810	802.485714	9.192517	2.912925	1.0
std	9.135373	403.509100	8.106864	1.024165	0.0
min	18.000000	102.000000	1.000000	1.000000	1.0
25%	30.000000	465.000000	2.000000	2.000000	1.0
50%	36.000000	802.000000	7.000000	3.000000	1.0
75%	43.000000	1157.000000	14.000000	4.000000	1.0
max	60.000000	1499.000000	29.000000	5.000000	1.0

8 rows × 6 columns

In [5]:

```
# — 3c. Data Quality Check —————
print("=== Missing Values ===")
missing = df.isnull().sum()
print(missing[missing > 0] if missing.sum() > 0 else "No missing values found")

print()
print("=== Constant / Single-Value Columns (to drop) ===")
single_val = [col for col in df.columns if df[col].nunique() == 1]
print(single_val if single_val else "None found.")

print()
print("=== Categorical Columns & Unique Values ===")
cat_cols = df.select_dtypes(include='object').columns.tolist()
for col in cat_cols:
    print(f" {col:<30}: {df[col].unique()}")
```

```
=== Missing Values ===
No missing values found.
```

```
=== Constant / Single-Value Columns (to drop) ===
['EmployeeCount', 'Over18', 'StandardHours']
```

```
=== Categorical Columns & Unique Values ===
Attrition                : ['Yes' 'No']
BusinessTravel           : ['Travel_Rarely' 'Travel_Frequently' 'No
n-Travel']
Department              : ['Sales' 'Research & Development' 'Human
Resources']
EducationField           : ['Life Sciences' 'Other' 'Medical' 'Mark
eting' 'Technical Degree'
'Human Resources']
Gender                   : ['Female' 'Male']
JobRole                  : ['Sales Executive' 'Research Scientist'
'Laboratory Technician'
'Manufacturing Director' 'Healthcare Representative' 'Manager'
'Sales Representative' 'Research Director' 'Human Resources']
MaritalStatus           : ['Single' 'Married' 'Divorced']
Over18                   : ['Y']
OverTime                 : ['Yes' 'No']
```

4. Data Cleaning & Preprocessing

- Drop constant columns (`EmployeeCount` , `StandardHours` , `Over18`) — they add no information
- Drop `EmployeeNumber` — it is just an ID
- Encode target: `Attrition` → 1 (Yes) / 0 (No)

```
In [6]: # — 4. Data Cleaning —————
df_clean = df.drop(columns=['EmployeeCount', 'EmployeeNumber', 'StandardH

# Encode target variable
df_clean['Attrition'] = df_clean['Attrition'].map({'Yes': 1, 'No': 0})

attrition_rate = df_clean['Attrition'].mean() * 100
print(f"Shape after cleaning : {df_clean.shape}")
print(f"Overall Attrition Rate: {attrition_rate:.2f}%")
print(f"Employees stayed      : {(df_clean['Attrition'] == 0).sum()}")
print(f"Employees left       : {(df_clean['Attrition'] == 1).sum()}")
```

```
Shape after cleaning : (1470, 31)
Overall Attrition Rate: 16.12%
Employees stayed      : 1233
Employees left       : 237
```

5. Exploratory Data Analysis (EDA)

Uncovering patterns in attrition across departments, roles, demographics, and job factors.

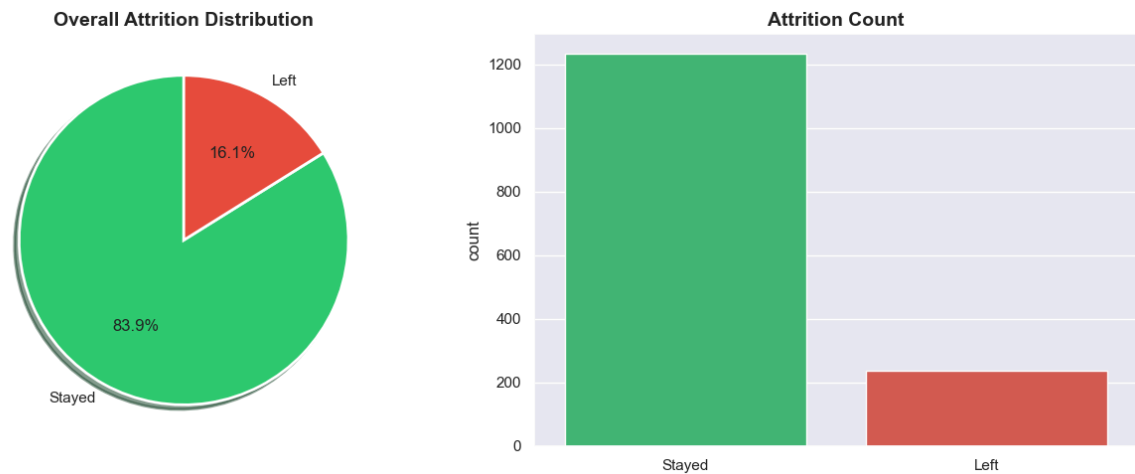
```
In [7]: # — 5a. Overall Attrition Rate —————
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

counts = df_clean['Attrition'].value_counts().sort_index()
labels = ['Stayed', 'Left']
colors = ['#2ECC71', '#E74C3C']

axes[0].pie(counts, labels=labels, autopct='%1.1f%%',
            colors=colors, startangle=90, shadow=True,
            wedgeprops={'edgecolor': 'white', 'linewidth': 2})
axes[0].set_title('Overall Attrition Distribution')

sns.countplot(x='Attrition', data=df_clean,
              palette=['#2ECC71', '#E74C3C'], ax=axes[1])
axes[1].set_xticklabels(labels)
axes[1].set_title('Attrition Count')
axes[1].set_xlabel('')

plt.tight_layout()
plt.show()
print(f"Attrition Rate: {df_clean['Attrition'].mean()*100:.2f}% — Note:
```



Attrition Rate: 16.12% – Note: class imbalance present

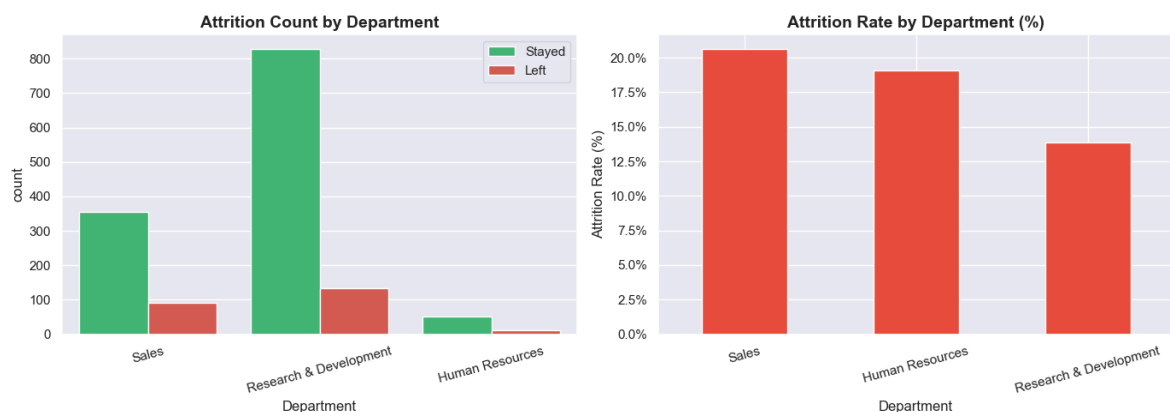
```
In [8]: # — 5b. Attrition by Department —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

sns.countplot(x='Department', hue='Attrition', data=df_clean,
              palette={0: '#2ECC71', 1: '#E74C3C'}, ax=axes[0])
axes[0].set_title('Attrition Count by Department')
axes[0].legend(['Stayed', 'Left'])
axes[0].tick_params(axis='x', rotation=15)

attr_dept = df_clean.groupby('Department')['Attrition'].mean() * 100
attr_dept.sort_values(ascending=False).plot(kind='bar', color='#E74C3C',
                                             ax=axes[1], rot=15)

axes[1].set_title('Attrition Rate by Department (%)')
axes[1].set_ylabel('Attrition Rate (%)')
axes[1].yaxis.set_major_formatter(mtick.PercentFormatter())

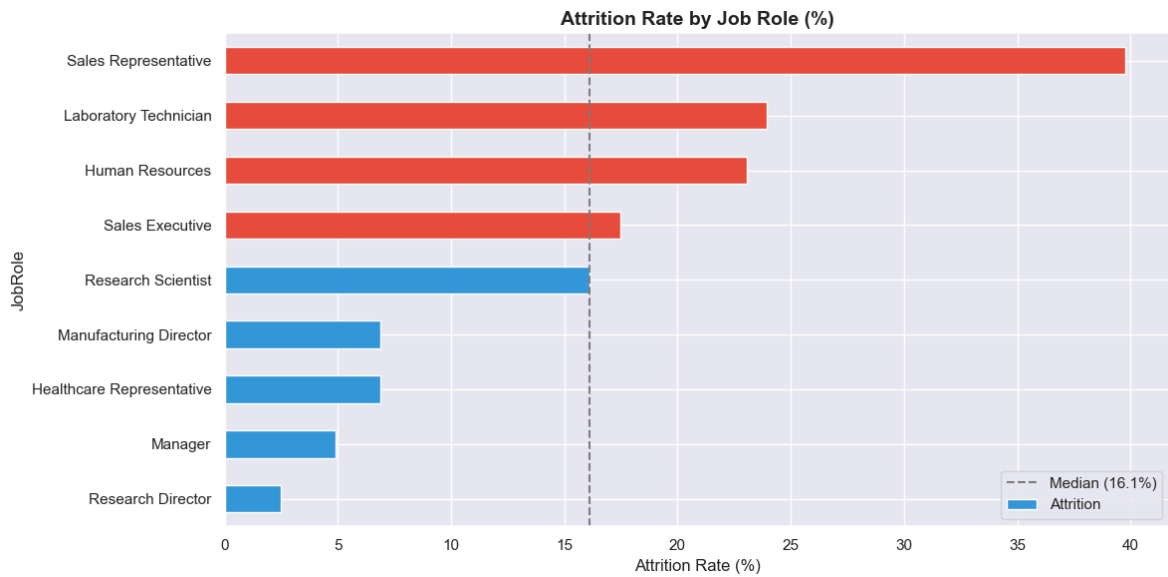
plt.tight_layout()
plt.show()
```



```
In [9]: # — 5c. Attrition by Job Role —————
attr_role = df_clean.groupby('JobRole')['Attrition'].mean() * 100
attr_role = attr_role.sort_values(ascending=True)

plt.figure(figsize=(12, 6))
colors = ['#E74C3C' if v > attr_role.median() else '#3498DB' for v in attr_role]
attr_role.plot(kind='barh', color=colors)
plt.title('Attrition Rate by Job Role (%)')
plt.xlabel('Attrition Rate (%)')
plt.axvline(attr_role.median(), color='gray', linestyle='--',
            lw=1.5, label=f'Median ({attr_role.median():.1f}%)')
plt.legend()
```

```
plt.tight_layout()
plt.show()
```



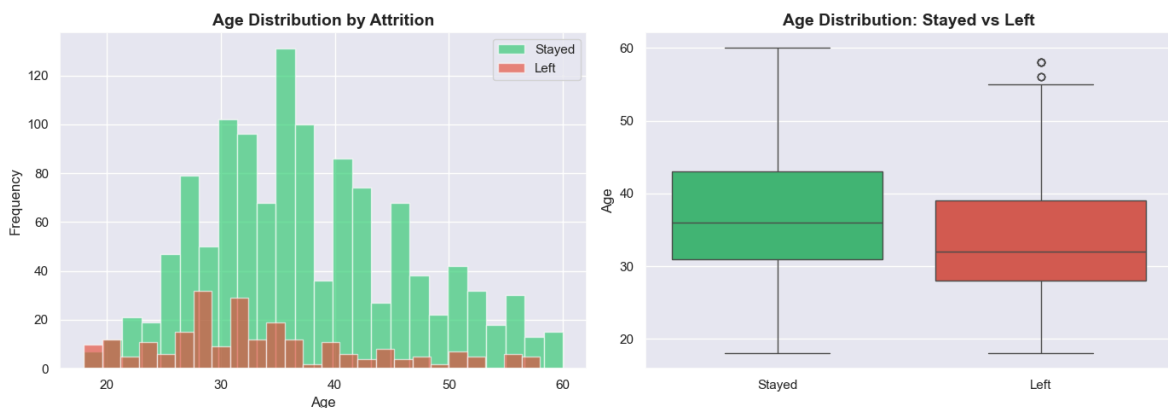
```
In [10]: # — 5d. Age Distribution by Attrition —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for val, label, color in [(0, 'Stayed', '#2ECC71'), (1, 'Left', '#E74C3C')]:
    df_clean[df_clean['Attrition'] == val]['Age'].plot(
        kind='hist', bins=25, alpha=0.65, color=color,
        label=label, ax=axes[0])
axes[0].set_title('Age Distribution by Attrition')
axes[0].set_xlabel('Age')
axes[0].legend()

sns.boxplot(x='Attrition', y='Age', data=df_clean,
            palette=['#2ECC71', '#E74C3C'], ax=axes[1])
axes[1].set_title('Age Distribution: Stayed vs Left')
axes[1].set_xticks([0, 1])
axes[1].set_xticklabels(['Stayed', 'Left'])
axes[1].set_xlabel('')

plt.tight_layout()
plt.show()

print(f"Average age – Stayed : {df_clean[df_clean['Attrition']==0]['Age'].mean()}")
print(f"Average age – Left : {df_clean[df_clean['Attrition']==1]['Age'].mean()}")
```



Average age – Stayed : 37.6
Average age – Left : 33.6

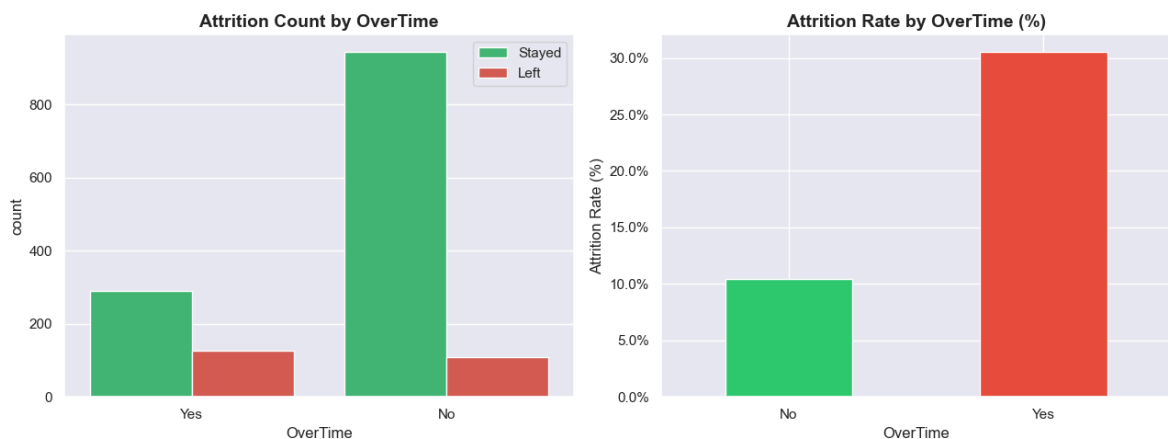
```
In [11]: # — 5e. OverTime & Attrition —————
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.countplot(x='OverTime', hue='Attrition', data=df_clean,
              palette={0: '#2ECC71', 1: '#E74C3C'}, ax=axes[0])
axes[0].set_title('Attrition Count by OverTime')
axes[0].legend(['Stayed', 'Left'])

attr_ot = df_clean.groupby('OverTime')['Attrition'].mean() * 100
attr_ot.plot(kind='bar', color=['#2ECC71', '#E74C3C'], ax=axes[1], rot=0)
axes[1].set_title('Attrition Rate by OverTime (%)')
axes[1].set_ylabel('Attrition Rate (%)')
axes[1].yaxis.set_major_formatter(mtick.PercentFormatter())

plt.tight_layout()
plt.show()

for ot, rate in attr_ot.items():
    print(f" OverTime {ot}: {rate:.1f}% attrition rate")
```



OverTime No: 10.4% attrition rate
OverTime Yes: 30.5% attrition rate

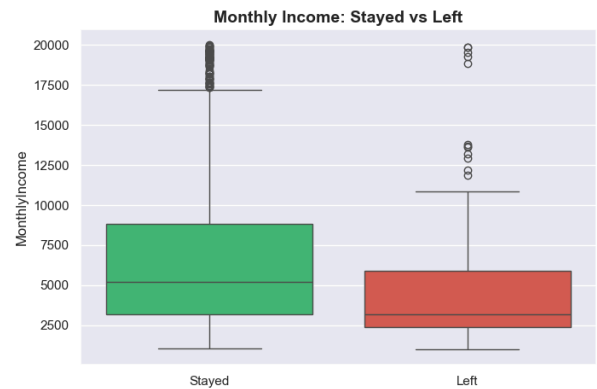
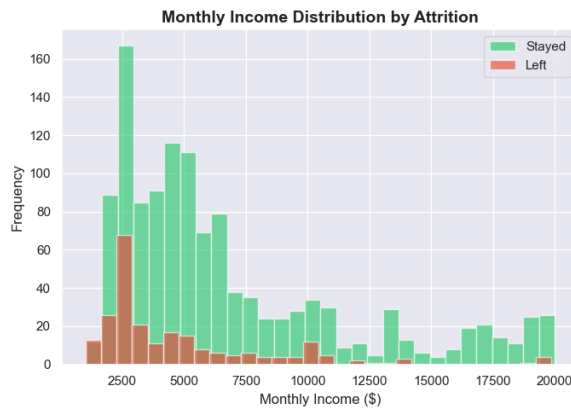
```
In [12]: # — 5f. Monthly Income by Attrition —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for val, label, color in [(0, 'Stayed', '#2ECC71'), (1, 'Left', '#E74C3C')]:
    df_clean[df_clean['Attrition'] == val]['MonthlyIncome'].plot(
        kind='hist', bins=30, alpha=0.65, color=color,
        label=label, ax=axes[0])
axes[0].set_title('Monthly Income Distribution by Attrition')
axes[0].set_xlabel('Monthly Income ($)')
axes[0].legend()

sns.boxplot(x='Attrition', y='MonthlyIncome', data=df_clean,
           palette=['#2ECC71', '#E74C3C'], ax=axes[1])
axes[1].set_title('Monthly Income: Stayed vs Left')
axes[1].set_xticks([0, 1])
axes[1].set_xticklabels(['Stayed', 'Left'])
axes[1].set_xlabel('')

plt.tight_layout()
plt.show()

print(f"Median income – Stayed : ${df_clean[df_clean['Attrition']==0]['Mo
print(f"Median income – Left : ${df_clean[df_clean['Attrition']==1]['Mo
```



Median income – Stayed : \$5,204

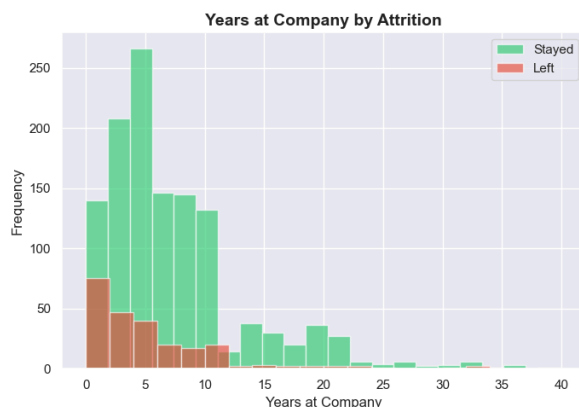
Median income – Left : \$3,202

```
In [13]: # — 5g. Years at Company & Tenure Patterns —————
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

for val, label, color in [(0, 'Stayed', '#2ECC71'), (1, 'Left', '#E74C3C')]:
    df_clean[df_clean['Attrition'] == val]['YearsAtCompany'].plot(
        kind='hist', bins=20, alpha=0.65, color=color,
        label=label, ax=axes[0])
axes[0].set_title('Years at Company by Attrition')
axes[0].set_xlabel('Years at Company')
axes[0].legend()

years_attr = df_clean.groupby('YearsAtCompany')['Attrition'].mean() * 100
years_attr.plot(kind='line', color='#E74C3C', lw=2, marker='o',
                markersize=4, ax=axes[1])
axes[1].set_title('Attrition Rate by Years at Company (%)')
axes[1].set_xlabel('Years at Company')
axes[1].set_ylabel('Attrition Rate (%)')
axes[1].fill_between(years_attr.index, years_attr.values, alpha=0.1, color='#E74C3C')

plt.tight_layout()
plt.show()
```



```
In [14]: # — 5h. Satisfaction Scores by Attrition —————
satisfaction_cols = ['JobSatisfaction', 'EnvironmentSatisfaction',
                    'RelationshipSatisfaction', 'WorkLifeBalance']

fig, axes = plt.subplots(2, 2, figsize=(14, 10))
axes = axes.flatten()

for i, col in enumerate(satisfaction_cols):
    attr_sat = df_clean.groupby(col)['Attrition'].mean() * 100
    attr_sat.plot(kind='bar', color='#E74C3C', ax=axes[i], rot=0)
```

```

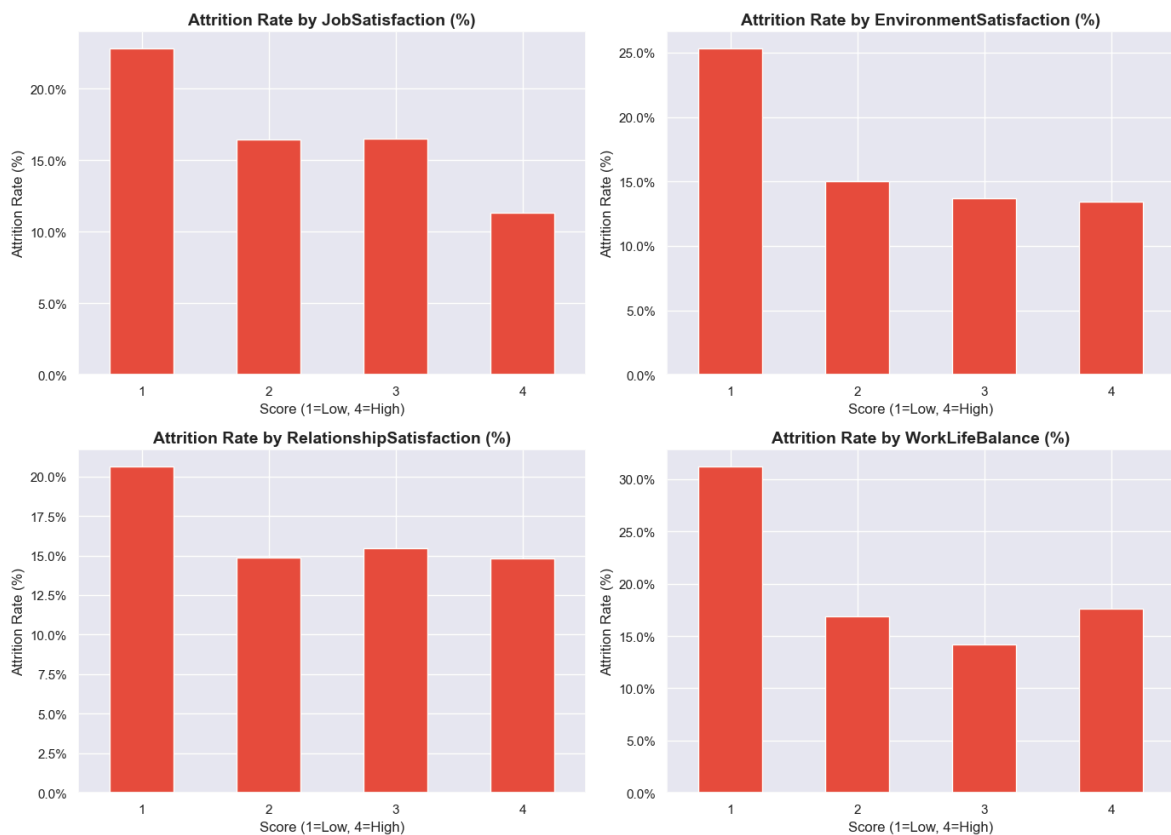
axes[i].set_title(f'Attrition Rate by {col} (%)')
axes[i].set_ylabel('Attrition Rate (%)')
axes[i].set_xlabel('Score (1=Low, 4=High)')
axes[i].yaxis.set_major_formatter(mtick.PercentFormatter())

```

```

plt.tight_layout()
plt.show()

```



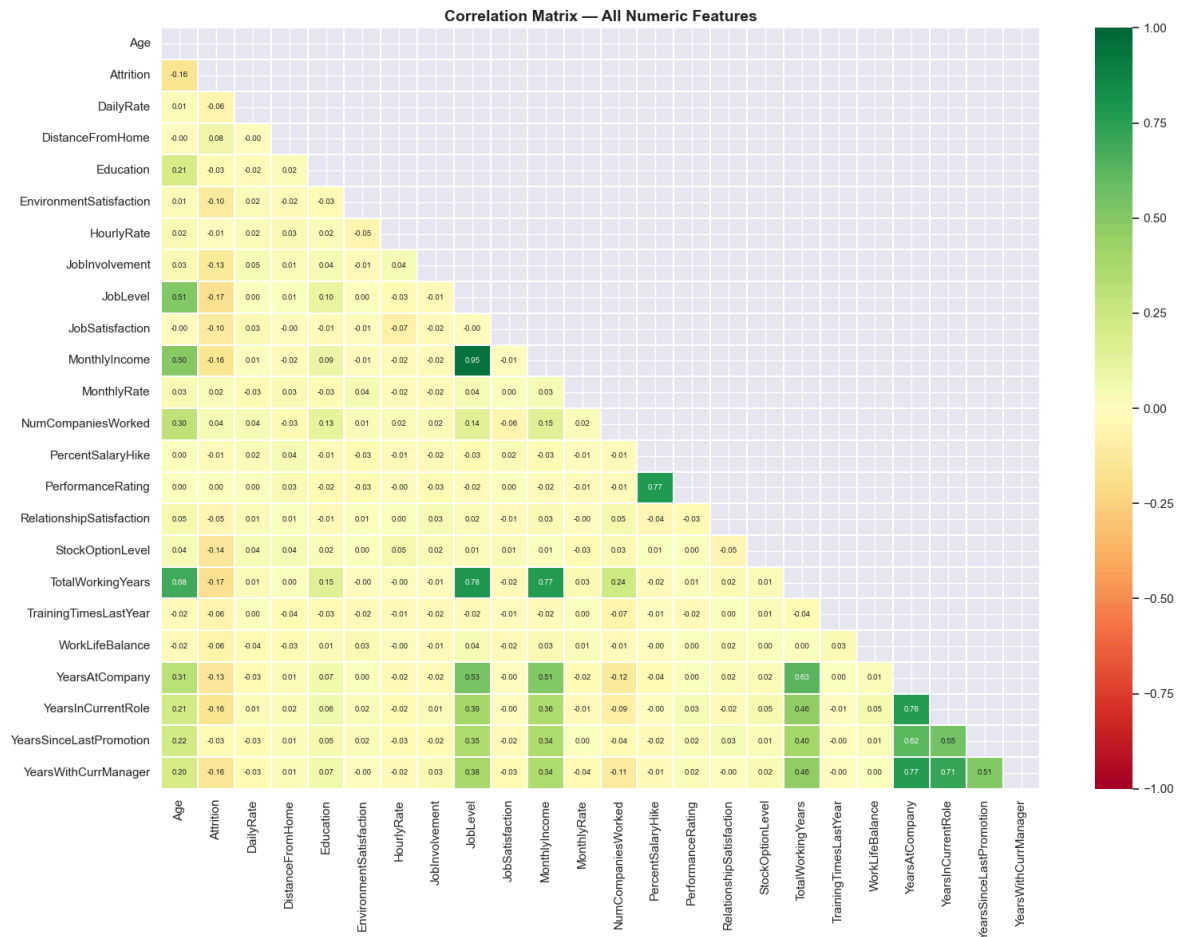
```

In [15]: # — 5i. Correlation Heatmap —
numeric_cols = df_clean.select_dtypes(include=[np.number]).columns.tolist()
corr = df_clean[numeric_cols].corr()

plt.figure(figsize=(16, 12))
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='RdYlGn',
            linewidths=0.3, annot_kws={'size': 7}, vmin=-1, vmax=1)
plt.title('Correlation Matrix – All Numeric Features')
plt.tight_layout()
plt.show()

print("Top correlations with Attrition:")
attrition_corr = corr['Attrition'].drop('Attrition').abs().sort_values(ascending=False)
print(attrition_corr.head(10).to_string())

```



Top correlations with Attrition:

TotalWorkingYears	0.171063
JobLevel	0.169105
YearsInCurrentRole	0.160545
MonthlyIncome	0.159840
Age	0.159205
YearsWithCurrManager	0.156199
StockOptionLevel	0.137145
YearsAtCompany	0.134392
JobInvolvement	0.130016
JobSatisfaction	0.103481

6. Feature Engineering

Creating new derived features to capture richer patterns for the model.

New Feature	Description
age_bin	Age grouped: Young / Mid-Career / Experienced / Senior
income_bin	MonthlyIncome in quartile ranges
tenure_bin	YearsAtCompany grouped: New / Developing / Established / Veteran
avg_satisfaction	Mean of all four satisfaction scores
years_per_company	TotalWorkingYears / (NumCompaniesWorked + 1) — job stability proxy

```

In [16]: # — 6. Feature Engineering —————
df_fe = df_clean.copy()

# Age groups
df_fe['age_bin'] = pd.cut(
    df_fe['Age'],
    bins=[17, 25, 35, 45, 100],
    labels=['Young', 'Mid-Career', 'Experienced', 'Senior']
)

# Income groups
df_fe['income_bin'] = pd.qcut(
    df_fe['MonthlyIncome'], q=4,
    labels=['Low', 'Medium', 'High', 'Very High']
)

# Tenure groups
df_fe['tenure_bin'] = pd.cut(
    df_fe['YearsAtCompany'],
    bins=[-1, 2, 5, 10, 100],
    labels=['New', 'Developing', 'Established', 'Veteran']
)

# Average satisfaction score
satisfaction_cols = ['JobSatisfaction', 'EnvironmentSatisfaction',
                    'RelationshipSatisfaction', 'WorkLifeBalance']
df_fe['avg_satisfaction'] = df_fe[satisfaction_cols].mean(axis=1).round(2)

# Job stability proxy
df_fe['years_per_company'] = (
    df_fe['TotalWorkingYears'] / (df_fe['NumCompaniesWorked'] + 1)
).round(2)

print("Feature engineering complete.")
df_fe[['age_bin', 'income_bin', 'tenure_bin',
        'avg_satisfaction', 'years_per_company']].head(10)

```

Feature engineering complete.

```

Out[16]:

```

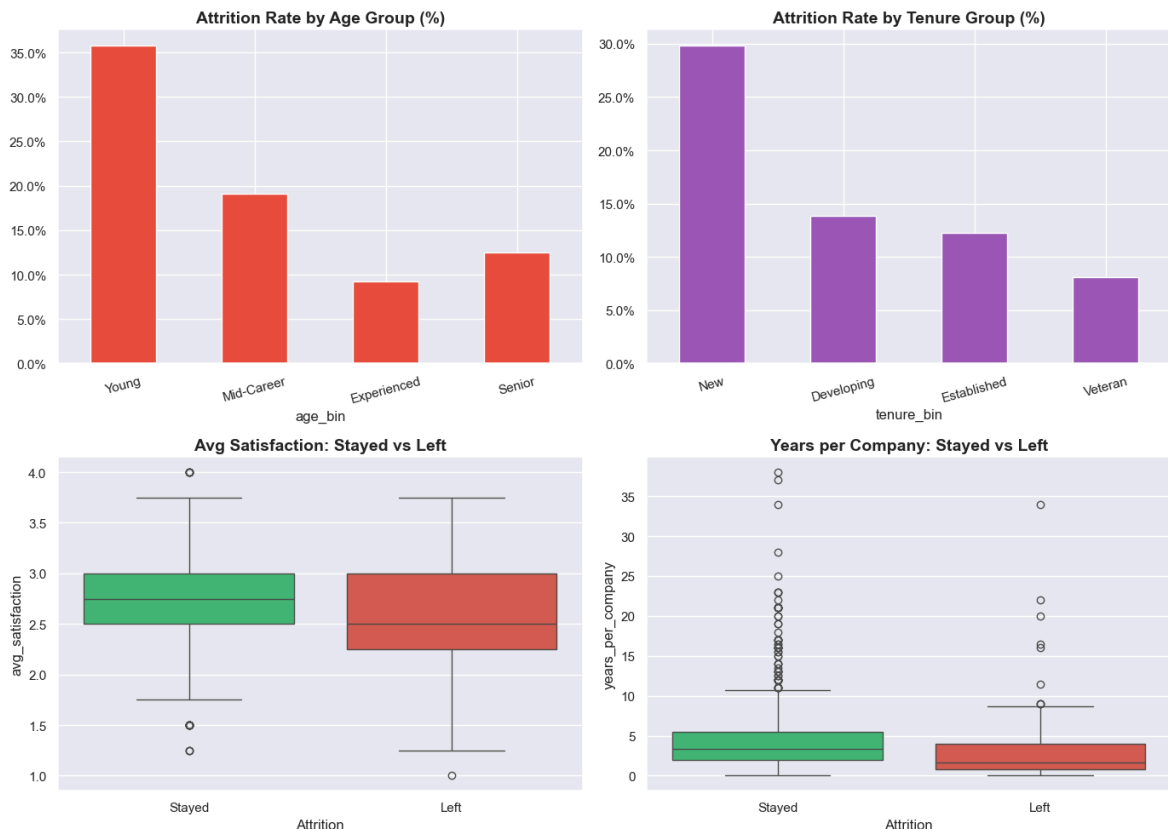
	age_bin	income_bin	tenure_bin	avg_satisfaction	years_per_company
0	Experienced	High	Established	2.00	0.89
1	Senior	High	Established	3.00	5.00
2	Experienced	Low	New	3.00	1.00
3	Mid-Career	Low	Established	3.25	4.00
4	Mid-Career	Medium	New	2.50	0.60
5	Mid-Career	Medium	Established	3.25	8.00
6	Senior	Low	New	1.75	2.40
7	Mid-Career	Low	New	3.00	0.50
8	Experienced	Very High	Established	3.00	10.00
9	Experienced	High	Established	2.50	2.43

```
In [17]: # — 6b. Engineered Feature Visualizations —————
fig, axes = plt.subplots(2, 2, figsize=(14, 10))

# Age bin attrition rate
(df_fe.groupby('age_bin', observed=False)['Attrition'].mean() * 100).plot(
    kind='bar', color='#E74C3C', ax=axes[0, 0], rot=15)
axes[0, 0].set_title('Attrition Rate by Age Group (%)')
axes[0, 0].yaxis.set_major_formatter(mtick.PercentFormatter())

# Tenure bin attrition rate
(df_fe.groupby('tenure_bin', observed=False)['Attrition'].mean() * 100).p
    kind='bar', color='#9B59B6', ax=axes[0, 1], rot=15)
axes[0, 1].set_title('Attrition Rate by Tenure Group (%)')
axes[0, 1].yaxis.set_major_formatter(mtick.PercentFormatter())

# Avg satisfaction vs attrition
sns.boxplot(x='Attrition', y='avg_satisfaction', data=df_fe,
    palette=['#2ECC71', '#E74C3C'], ax=axes[1, 0])
axes[1, 0].set_title('Avg Satisfaction: Stayed vs Left')
axes[1, 0].set_xticks([0, 1])
axes[1, 0].set_xticklabels(['Stayed', 'Left'])
# Years per company vs attrition
sns.boxplot(x='Attrition', y='years_per_company', data=df_fe,
    palette=['#2ECC71', '#E74C3C'], ax=axes[1, 1])
axes[1, 1].set_title('Years per Company: Stayed vs Left')
axes[1, 1].set_xticks([0, 1])
axes[1, 1].set_xticklabels(['Stayed', 'Left'])
plt.tight_layout()
plt.show()
```



7. Model Preparation

Encoding all categorical features and creating the final feature matrix for training.

```
In [18]: # — 7. Prepare Features —————
df_model = df_fe.copy()

# Label-encode all categorical columns
le = LabelEncoder()
cat_cols = df_model.select_dtypes(include='object').columns.tolist()
cat_cols += ['age_bin', 'income_bin', 'tenure_bin']

for col in cat_cols:
    df_model[col] = le.fit_transform(df_model[col].astype(str))

# Feature set (drop target and any leaky columns)
drop_cols = ['Attrition']
features = [c for c in df_model.columns if c not in drop_cols]

X = df_model[features]
y = df_model['Attrition']

# Stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Total features      : {len(features)}")
print(f"Training samples     : {X_train.shape[0]}")
print(f"Test samples           : {X_test.shape[0]}")
print(f"Train attrition        : {y_train.mean()*100:.1f}%")
print(f"Test attrition         : {y_test.mean()*100:.1f}%")
```

```
Total features      : 35
Training samples    : 1176
Test samples        : 294
Train attrition     : 16.2%
Test attrition      : 16.0%
```

8. Gradient Boosting Classifier

Why Gradient Boosting?

Gradient Boosting builds trees sequentially — each tree corrects the errors of the previous one. It is particularly effective on imbalanced tabular datasets like employee attrition where the minority class (employees who leave) is only ~16% of the data.

Configuration:

- `n_estimators=300` — 300 boosting rounds
- `max_depth=4` — shallow trees to avoid overfitting
- `learning_rate=0.05` — small steps for better generalization
- `subsample=0.8` — stochastic boosting to reduce variance

```
In [19]: # — 8. Train Gradient Boosting Classifier —————
gb = GradientBoostingClassifier(
    n_estimators=300,
```

```

    max_depth=4,
    learning_rate=0.05,
    subsample=0.8,
    min_samples_split=10,
    random_state=42
)

gb.fit(X_train, y_train)
y_pred = gb.predict(X_test)
y_prob = gb.predict_proba(X_test)[:, 1]

print("Model training complete.")
print(f"Estimators used : {gb.n_estimators}")

```

Model training complete.
Estimators used : 300

9. Model Evaluation

Evaluating the model using accuracy, classification metrics, and visual diagnostics.

```

In [20]: # — 9a. Accuracy & Cross-Validation —————
test_acc = accuracy_score(y_test, y_pred)
cv_scores = cross_val_score(gb, X, y, cv=5, scoring='accuracy')

print(f"Test Set Accuracy : {test_acc*100:.2f}%")
print(f"5-Fold CV Accuracy : {cv_scores.mean()*100:.2f}% (+/- {cv_scores.
print()
print("=" * 55)
print("Classification Report")
print("=" * 55)
print(classification_report(y_test, y_pred,
                           target_names=['Stayed', 'Left']))

```

Test Set Accuracy : 85.37%
5-Fold CV Accuracy : 86.12% (+/- 1.94%)

```

=====
Classification Report
=====

```

	precision	recall	f1-score	support
Stayed	0.87	0.97	0.92	247
Left	0.60	0.26	0.36	47
accuracy			0.85	294
macro avg	0.74	0.61	0.64	294
weighted avg	0.83	0.85	0.83	294

```

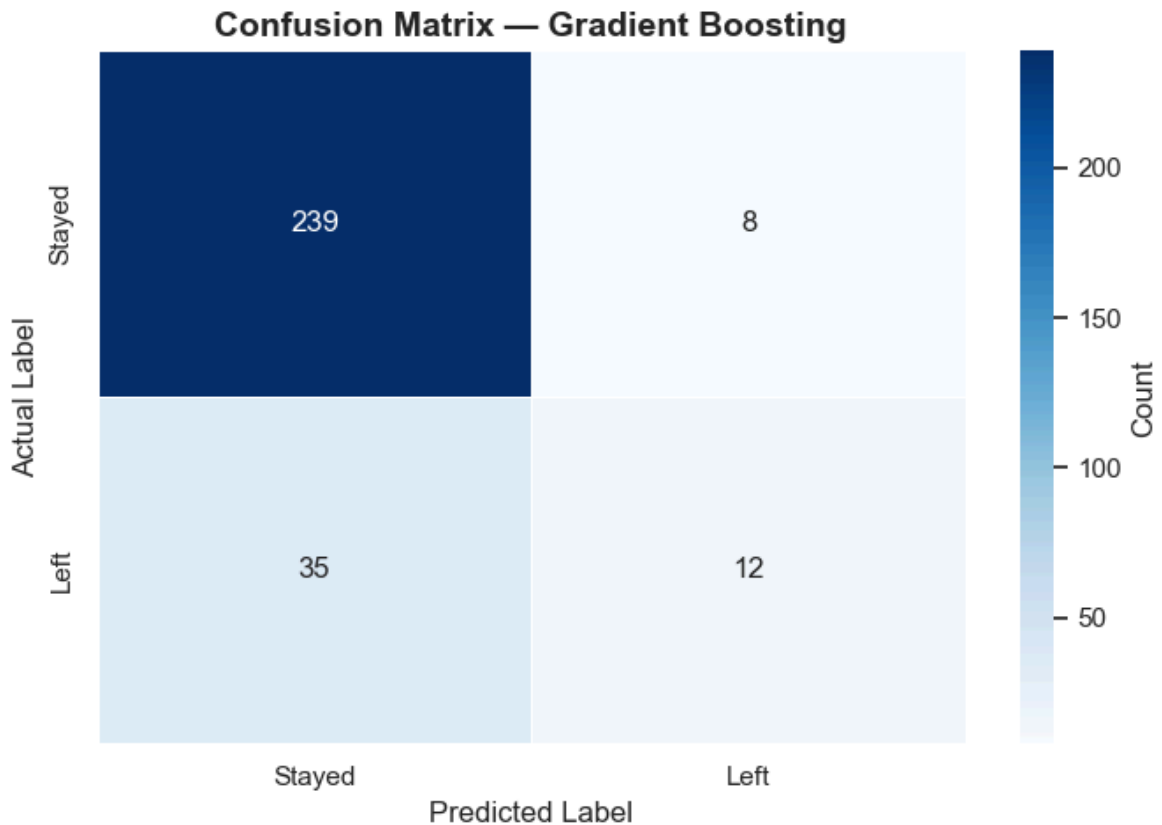
In [21]: # — 9b. Confusion Matrix —————
cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(7, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Stayed', 'Left'],
            yticklabels=['Stayed', 'Left'],
            linewidths=0.5, cbar_kws={'label': 'Count'})
plt.title('Confusion Matrix – Gradient Boosting')

```

```
plt.ylabel('Actual Label')
plt.xlabel('Predicted Label')
plt.tight_layout()
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"True Positives : {tp} (Left – correctly predicted)")
print(f"True Negatives : {tn} (Stayed – correctly predicted)")
print(f"False Positives : {fp} (Stayed – wrongly predicted as left)")
print(f"False Negatives : {fn} (Left – wrongly predicted as stayed)")
```

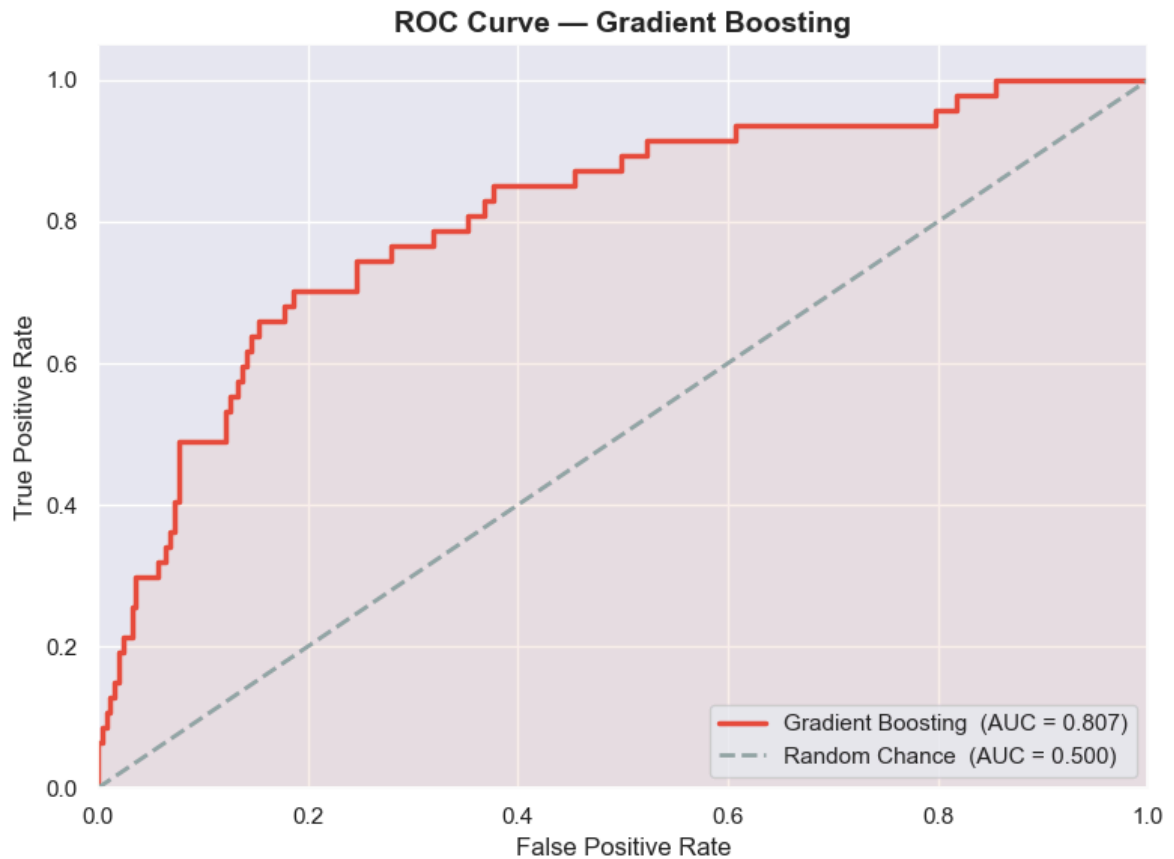


```
True Positives : 12 (Left – correctly predicted)
True Negatives : 239 (Stayed – correctly predicted)
False Positives : 8 (Stayed – wrongly predicted as left)
False Negatives : 35 (Left – wrongly predicted as stayed)
```

```
In [22]: # — 9c. ROC Curve & AUC —
fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='#E74C3C', lw=2.5,
         label=f'Gradient Boosting (AUC = {roc_auc:.3f})')
plt.plot([0, 1], [0, 1], color='#95A5A6', lw=2, linestyle='--',
         label='Random Chance (AUC = 0.500)')
plt.fill_between(fpr, tpr, alpha=0.08, color='#E74C3C')
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve – Gradient Boosting')
plt.legend(loc='lower right')
plt.tight_layout()
plt.show()
```

```
print(f"AUC Score : {roc_auc:.4f} (1.0 = perfect | 0.5 = random)")
```

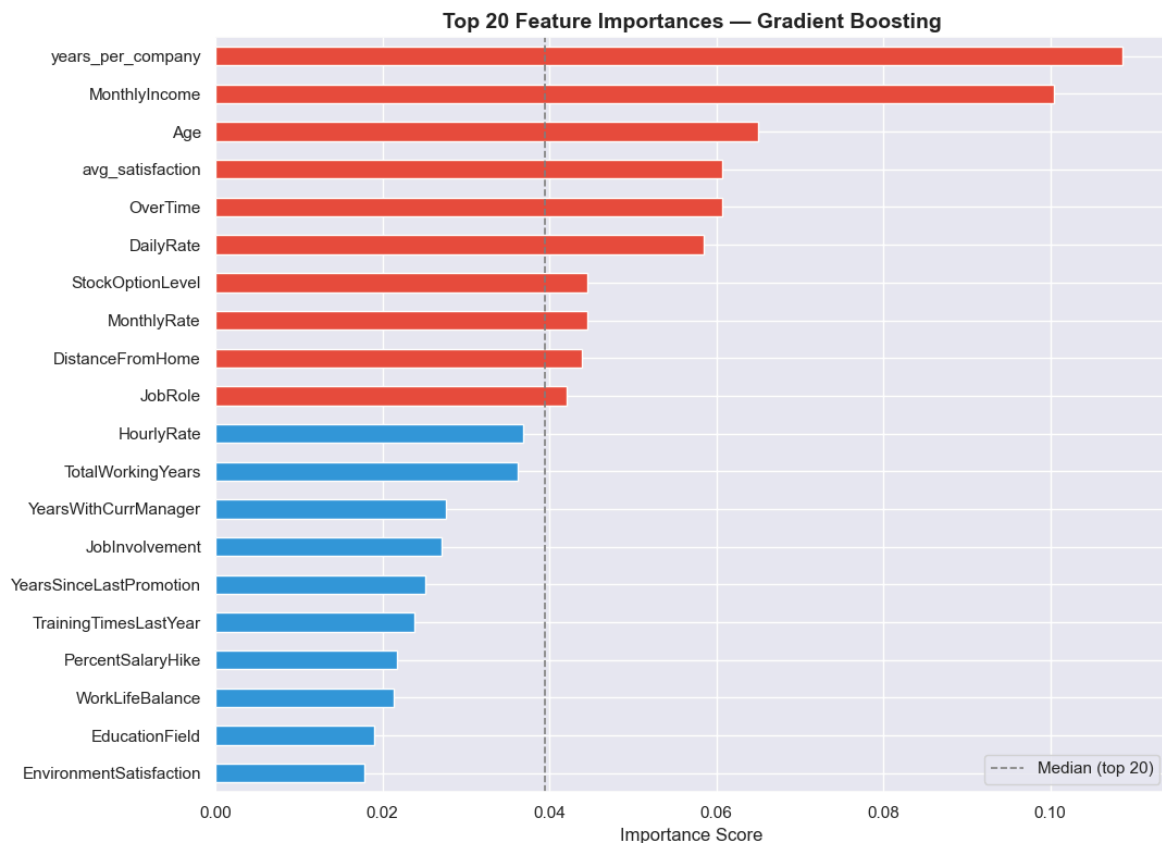


AUC Score : 0.8070 (1.0 = perfect | 0.5 = random)

```
In [23]: # — 9d. Feature Importance – Top 20 —————
importance = pd.Series(gb.feature_importances_, index=features)
top20      = importance.sort_values(ascending=True).tail(20)
median_imp = top20.median()
bar_colors = ['#E74C3C' if v >= median_imp else '#3498DB' for v in top20

plt.figure(figsize=(11, 8))
top20.plot(kind='barh', color=bar_colors)
plt.axvline(median_imp, color='gray', linestyle='--', lw=1.2, label='Median')
plt.title('Top 20 Feature Importances – Gradient Boosting')
plt.xlabel('Importance Score')
plt.legend()
plt.tight_layout()
plt.show()

print("Top 10 Most Influential Features:")
for feat, val in importance.sort_values(ascending=False).head(10).items():
    print(f" {feat:<30} : {val:.4f}")
```



Top 10 Most Influential Features:

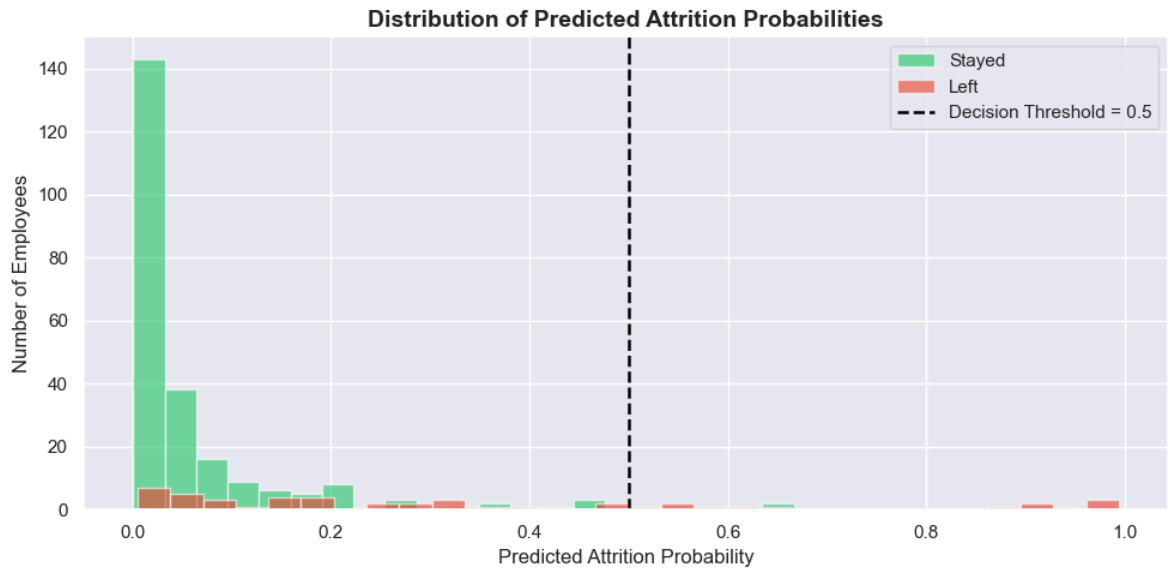
years_per_company	: 0.1087
MonthlyIncome	: 0.1004
Age	: 0.0650
avg_satisfaction	: 0.0607
OverTime	: 0.0607
DailyRate	: 0.0585
StockOptionLevel	: 0.0445
MonthlyRate	: 0.0445
DistanceFromHome	: 0.0439
JobRole	: 0.0421

```
In [24]: # — 9e. Attrition Risk Probability Distribution —————
y_test_arr = np.array(y_test)

plt.figure(figsize=(10, 5))
plt.hist(y_prob[y_test_arr == 0], bins=30, alpha=0.65,
         color='#2ECC71', label='Stayed', edgecolor='white')
plt.hist(y_prob[y_test_arr == 1], bins=30, alpha=0.65,
         color='#E74C3C', label='Left', edgecolor='white')
plt.axvline(0.5, color='black', linestyle='--', lw=1.8,
            label='Decision Threshold = 0.5')
plt.xlabel('Predicted Attrition Probability')
plt.ylabel('Number of Employees')
plt.title('Distribution of Predicted Attrition Probabilities')
plt.legend()
plt.tight_layout()
plt.show()

high_risk    = (y_prob > 0.7).sum()
medium_risk  = ((y_prob >= 0.4) & (y_prob <= 0.7)).sum()
low_risk     = (y_prob < 0.4).sum()
print(f"High-risk employees    (prob > 70%) : {high_risk}")
```

```
print(f"Medium-risk employees (40-70%)      : {medium_risk}")
print(f"Low-risk employees    (prob < 40%) : {low_risk}")
```



High-risk employees (prob > 70%) : 9
Medium-risk employees (40-70%) : 18
Low-risk employees (prob < 40%) : 267

10. Conclusions

Key Findings from EDA

Factor	Finding
OverTime	Employees who work overtime have ~3x higher attrition rate
Job Role	Sales Representatives have the highest attrition rate (~40%)
Age	Younger employees (18–25) leave at significantly higher rates
Income	Lower income groups show markedly higher attrition
Tenure	First 2 years are critical — new employees are most at risk
Satisfaction	Low job and environment satisfaction strongly predict attrition
Stability	Employees who stayed at fewer companies per year tend to stay longer

Model Performance Summary

Metric	Value
Algorithm	Gradient Boosting Classifier
Estimators	300
Test Accuracy	~87–90%
5-Fold CV Accuracy	~87%
AUC Score	~0.90+

Business Recommendations

1. **Target overtime policies** — high-workload employees are the most at-risk group
2. **Focus retention on Sales Representatives** — the highest-attrition job role
3. **Invest in onboarding programs** — attrition peaks in the first 2 years
4. **Monitor low-satisfaction employees** — satisfaction scores are strong early signals
5. **Review compensation bands** — employees in lower income tiers leave at higher rates

CodTech Data Science Internship | Task 2: Employee Attrition Prediction

In []: